Al-Qahera at AraSeg Shared Task 2026: Genre-Conditioned LLM Prompting versus Supervised Multi-Backbone Ensembling

Mohamed Mohamed
American University in Cairo
Cairo, Egypt
mabdelmagid@aucegypt.edu

Hazem Ajlan
American University in Cairo
Cairo, Egypt
hazemdiaa@aucegypt.edu

Abstract

We submit systems to both the closed and open tracks of all four subtasks of the AraSeg 2026 shared task (PA, NP, NoPnx-PA, NoPnx-NP): a supervised 3-backbone ensemble (CAMeLBERT-MSA, AraBERTv02, and AraELECTRA, each with a gated-residual head, R-Drop regularization, and stochastic weight averaging), and a zero-shot LLM system prompted with explicit, subtask-specific segmentation rules instead of labeled examples. The ensemble uses no external resources, so it is entered in both tracks everywhere; for NoPnx-NP specifically, where every structural cue is stripped away, we additionally blend its predictions with the LLM system's, in a real-data-only variant (closed-eligible) and a wiki-augmented variant trained with added pseudo-labeled Wikipedia data (open-only). On the officially scored Blind Test set, our best submission per subtask reaches 94.70 F_1 (PA), 91.00 (NP), 91.20 (NoPnx-PA), and 87.40 (NoPnx-NP). The supervised ensemble is the stronger, more consistent choice overall; the one exception is NoPnx-PA, where our LLM system's unmodified seed prompt outperforms it and combining the two systems does not help – the only subtask where that happens. We report in full the structural annotation ambiguities we uncovered while building the LLM system.

1 Introduction

This paper describes the systems submitted by team Al-Qahera to the AraSeg 2026 shared task (Elkholy et al., 2026), covering all four subtasks (PA, NP, NoPnx-PA, NoPnx-NP) in both tracks. Sentence segmentation is normally solved once punctuation is available; AraSeg 2026 isolates exactly the settings where it is not: Modern Standard Arabic text with punctuation removed, paragraph breaks removed, or both. Under these conditions a boundary is not a token-level cue but a judgment about where one structural unit (a sentence, a Qur'ānic āyah, a quiz option, a poem line) ends and the next begins, and that judgment is genre-dependent – the same conjunction class (و/ف/ثم, *wa*/*fa*/*thumma*, roughly "and"/"so"/"then") is a continuer in legal prose and a sentence-opener in expository essays; Appendix A quantifies this directly with a genre-stratified breakdown, including a minimal pair on identical documents with only the visible cues changed.

We submit two systems reflecting two ways of making that judgment: (i) a supervised encoder ensemble learning boundary statistics from labeled examples (Section 3.1), and (ii) a zero-shot LLM system prompted with explicit segmentation rules (Section 3.3), combined with (i) for NoPnx-NP's open track (Section 3.2). Neither dominates uniformly: the LLM system needed extensive per-subtask prompt refinement to become competitive on the punctuation-visible subtasks, and trails the ensemble on dev/test except where every structural cue is stripped away (NoPnx-NP). We report this pattern, and the annotation ambiguities we uncovered building the LLM system, in full.

2 Background

Task. AraSeg 2026 is formulated as binary per-token classification: given a document as an ordered token sequence, predict whether a sentence boundary follows each token. Four subtask variants cross two binary factors – punctuation visible or not, paragraph breaks visible or not – giving PA (both visible), NP (punctuation only), NoPnx-PA (paragraph breaks only), and NoPnx-NP (neither). Critically, PA and NoPnx-PA are two renderings of the *same* 100-document pool, and NP and NoPnx-NP are two renderings of the *same* 212-document pool; only the visible cues differ. Official splits are shared across all four subtasks: 174 train, 222 dev, 262 test documents, plus a subtask-specific Blind Test set (100 docs for the PA/NoPnx-PA pool, 212 for the NP/NoPnx-NP pool). Team Al-Qahera en-

Proceedings of The Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026) - Volume 2 Shared Tasks, pages 1010–1021
October 28-29, 2026 ©2026 Association for Computational Linguistics

ters all four subtasks in both the closed and open tracks.

Genres. We use a 12-way genre taxonomy (essay, headers, lists, poetry, song, dialogue, reported speech, hadith, Qur'ān, Bible, legal, quiz) that we found necessary to condition both systems on, since the definition of "one unit" varies sharply by genre – a quiz answer option is always its own unit however short, a Qur'ānic āyah overrides ordinary clause boundaries in both directions, and legal prose fuses long coordinate clauses that would be split in expository prose.

Related work. Elkholy et al. (2026) introduce the AraSeg 2026 task and corpus and report that even a strong instruction-tuned LLM (Gemini 3.1 Pro) reaches only 76.8 F_1 on PA under naive prompting, well below a trained baseline – motivating the explicit segmentation rules in our own prompts. Our supervised architecture builds on CAMeLBERT (Inoue et al., 2021), AraBERTv02 (Antoun et al., 2020), and AraELECTRA (Antoun et al., 2021) as backbones, combined via R-Drop (Liang et al., 2021), stochastic weight averaging (Izmailov et al., 2018), and a Gated Residual Network head (Lim et al., 2021); confidence-weighted ensembles of the same family of Arabic transformers proved similarly effective for a concurrent ArabicNLP 2025 shared task on readability assessment (Basem et al., 2025); an earlier single-model iteration also used biaffine scoring (Dozat and Manning, 2017), superseded by Section 3.1's ensemble. Our initial guideline-synthesis procedure followed the MATTER/MAMA cycle (Pustejovsky and Stubbs, 2012); currently submitted prompts were instead refined via GEPA (Agrawal et al., 2025), an automatic prompt-optimization method related to ProTeGi (Pryzant et al., 2023), plus manual fixes from error analysis.

3 System Overview

3.1 Supervised system

All four subtasks use the same recipe (Figure 1): three backbones – `CAMeL-Lab/bert-base-arabic-camelbert-msa`, `aubmindlab/bert-base-arabertv02`, and `aubmindlab/araelectra-base-discriminator` – each fitted independently with a Gated Residual Network head, trained with R-Drop (α=2) and class-weighted cross-entropy, and averaged via stochastic weight averaging over the top-3

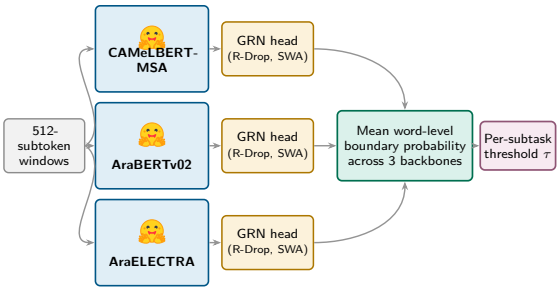


Figure 1: Supervised 3-backbone ensemble (Section 3.1): each backbone is fitted independently with a Gated Residual Network head, trained with R-Drop and stochastic weight averaging; the three backbones' word-level boundary probabilities are averaged and thresholded per subtask.

dev-F_1 checkpoints. Inference uses overlapping 512-subtoken windows (stride 32, mean-logit aggregation); the three backbones' boundary probabilities are averaged and thresholded per subtask via grid search on dev (PA 0.49, NP 0.33, NoPnx-PA 0.49, NoPnx-NP 0.41), replacing an earlier subtask-specific setup (a biaffine head for NoPnx-NP, test-time augmentation for PA, a dual-threshold scheme) that the plain ensemble matches or beats.

3.2 Open-track system for NoPnx-NP

NoPnx-NP is the only subtask where we found a competitive path to using external data. We pseudo-label streamed Arabic Wikipedia articles with our PA-subtask teacher (Section 3.1's CAMeLBERT-MSA+GRN model, run on punctuation-intact text), strip punctuation and diacritics to match NoPnx-NP's format, and transfer any boundary on a stripped token back onto the preceding word (Figure 2). Articles are keyword-bucketed by title/lead text into five coarse genres (literary, political, education, children, general) with a per-bucket quota – coarser than Section 2's 12 genres, since that is a granularity a keyword filter can approximate at Wikipedia scale; we did not audit bucketing accuracy or the teacher's label quality on this out-of-domain text, so some pseudo-label noise should be assumed. We sample 2500 pseudo-labeled documents and combine them with the 174 gold training documents to retrain a 2-backbone version of the recipe above (AraBERTv02 and AraELECTRA, dropping CAMeLBERT-MSA).

We combine two of the three backbones' predictions (AraBERTv02 and AraELECTRA, each thresholded at 0.5, dropping CAMeLBERT-MSA) with our LLM system's (Section 3.3) via a token-level rule: where the two backbones agree, keep

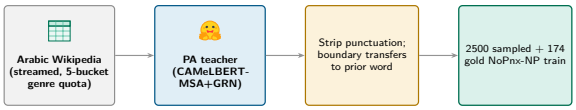


Figure 2: NoPnx-NP open-track data generation (Section 3.2): the PA-subtask teacher (a Hugging Face checkpoint, Section 3.1) labels punctuation-intact Wikipedia text, boundaries survive punctuation stripping via transfer-to-prior-word, and the result is combined with the gold training set.

their agreed label; where they disagree, use the LLM's prediction as tiebreak. This combined system is our open-track entry for NoPnx-NP: it reaches our best dev/test F_1 on this subtask (Table 1), though on the Blind Test set it is edged out by 0.10 points by the real-data-only ensemble+LLM blend (Table 2), which needs no external data and so qualifies for the closed track as well.

3.3 LLM system

An earlier two-stage pipeline – stage 1 classified each document into one or two of 12 genres, stage 2 segmented using only that genre's rules from a single shared guideline (`SEGMENTATION_SKILL.md`), itself synthesized once by cross-validating 8 independent LLM readings of the gold-labeled training set – reached held-out dev F_1 of 88.9 (NoPnx-NP, gemini-3-flash-preview), 85.8 (PA, gemma-4-31b-it), 76.9 (NP, gemma-4-31b-it, single untuned pass), and 72.9 (NoPnx-PA, gemma-4-31b-it, single untuned pass); these are the weaker Gemma model this pipeline was built on, not the current gemini-3.6-flash system, which superseded it with the single-stage prompts in Table 2.

Our currently submitted systems instead use a *single* schema-constrained call per document to `google/gemini-3.6-flash` (Kaggle Model Proxy, plus direct API keys as extra workers for NoPnx-NP), with no separate genre-classification stage – each subtask's prompt encodes genre-specific rules directly. PA and NoPnx-PA use a flat checklist of phenomena (conjunction fusion, reported-speech attribution, hadith isnad chains, poetry/Qur'ānic/Biblical verse-ends, MCQ options, bibliographic blocks), calibrated to the fact that most true boundaries land on punctuation tokens. NoPnx-NP's prompt instead performs a two-level coarse-then-fine pass – first marking discourse-shift blocks, then judging clause-completeness boundaries within each – found by an automated GEPA (Agrawal et al., 2025) search over candidate prompts rather than written by us directly. As before, the model returns anchor word+approximate-position entries, resolved deterministically onto the token sequence via nearest-occurrence, substring-fallback matching (Figure 3).

That resolver required three structural fixes, discovered through genre-stratified error analysis on the punctuation-visible subtasks and unchanged since: gold labels can sit on a trailing punctuation token rather than the anchor word; two visually identical parenthetical patterns (verse citations vs. multiple-choice option markers) resolve in opposite directions; and comma-vs-period surface form is not by itself a reliable boundary-existence signal. Fixing these took the original PA pipeline's 6-document smoke-test F_1 from 7.7 to 86.4 and its held-out dev F_1 from a naive-prompting equivalent of near-zero to 85.8.

Subtask	Track	Dev F_1	Test F_1
PA	Closed & Open	94.77	94.23
NP	Closed & Open	92.96	92.47
NoPnx-PA	Closed & Open	86.34	86.53
NoPnx-NP	Closed	84.00	84.55
NoPnx-NP	Open (+wiki, +LLM)	**85.58**	**84.87**

Table 1: Supervised 3-backbone ensemble, all four subtasks. PA/NP/NoPnx-PA use no external resources, so the same submission qualifies for both tracks; NoPnx-NP's open-track row is the distinct hybrid system of Section 3.2.

4 Experimental Setup

Both systems are evaluated on the same official dev/test splits and, where noted, the official Blind Test set. Supervised hyperparameters: AdamW, learning rate 3×10^{-5}, linear warmup (10% of steps), batch size 8 with gradient accumulation 4, up to 30 epochs with early stopping (patience 4, $\Delta\geq0.001$). LLM hyperparameters: temperature 0, max output tokens 4000, structured JSON output validated against a schema, one API call per document. All F_1 scores are computed at the per-token boundary level using the shared task's own scoring convention. Single-backbone component ablations (head type, R-Drop, SWA) and seed variance are reported in Appendix B.

5 Results

Table 2 reports every officially scored Blind Test set submission with a confirmed generating system, in chronological order per subtask. Three patterns stand out. First, on PA the single-backbone model (94.20, no matching dev/test score measured) trails the 3-backbone ensemble (94.70) by only half a point on the Blind Test set – a modest ensembling

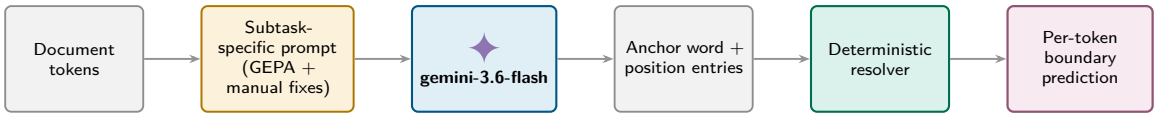


Figure 3: LLM system inference pipeline (Section 3.3), used identically across all four subtasks: a single schema-constrained call per document to `google/gemini-3.6-flash` returns anchor word+position boundary entries, resolved deterministically onto the token sequence.

Subtask	System	Blind F_1	Track
PA	Single-backbone CAMeLBERT-MSA (GRN+R-Drop)	94.20	Closed & Open
PA	LLM, single-stage (GEPA candidate)	85.20	Closed & Open
PA	LLM, single-stage (improved prompt)	90.30	Closed & Open
PA	Supervised ensemble (3-backbone)	**94.70**	Closed & Open
PA	LLM, single-stage (punctuation fix)	89.90	Closed & Open
NP	Single-backbone CAMeLBERT-MSA (baseline)	89.50	Closed & Open
NP	LLM, single-stage (GEPA-distilled prompt)	84.90	Closed & Open
NP	Supervised ensemble (3-backbone)	**91.00**	Closed & Open
NoPnx-PA	Single-backbone CAMeLBERT-MSA (GRN)	87.50	Closed & Open
NoPnx-PA	LLM, seed prompt (gemini-3-flash-preview)	88.40	Closed & Open
NoPnx-PA	LLM, seed prompt (gemini-3.6-flash)	**91.20**	Closed & Open
NoPnx-PA	Supervised ensemble (3-backbone)	89.70	Closed & Open
NoPnx-PA	Ensemble + LLM blend	90.60	Closed & Open
NoPnx-NP	Single model (CAMeLBERT-MSA+GRN+biaffine)	83.00	Closed & Open
NoPnx-NP	LLM, two-stage genre-classify (earlier pipeline)	84.60	Closed & Open
NoPnx-NP	LLM, single-stage flat prompt	85.90	Closed & Open
NoPnx-NP	LLM, hierarchical coarse-to-fine prompt	86.80	Closed & Open
NoPnx-NP	Supervised ensemble (3-backbone, real data only)	85.10	Closed & Open
NoPnx-NP	Ensemble (2-backbone, real data) + LLM blend	**87.40**	Closed & Open
NoPnx-NP	Ensemble (wiki-augmented, genre-filtered) + LLM blend	87.30	Open

Table 2: Every officially scored Blind Test set submission with confirmed provenance, in chronological order (bold = best per subtask). All rows qualify for both tracks except NoPnx-NP's wiki-augmented row (Section 3.2, Open-only, depends on external data). Two further, less certain NoPnx-NP wiki-blend submissions (87.00, 87.10) are omitted.

gain. Second, NoPnx-PA isolates a clean model-upgrade effect: the identical LLM seed prompt scored 88.40 on gemini-3-flash-preview and 91.20 after switching to gemini-3.6-flash, and that unmodified prompt remains our best NoPnx-PA result, ahead of the ensemble (89.70) and our ensemble+LLM blend (90.60) – the only subtask where combining did not help: our two voting backbones (Section 3.2) disagree with each other on just 1.2% of tokens, so the LLM tiebreak rarely fires and correlated backbone errors elsewhere survive into the blend. Third, NoPnx-NP shows the most iteration: from a single model (83.00) through three LLM prompting strategies (84.60–86.80) and a real-data ensemble (85.10), to a real-data ensemble+LLM blend at 87.40, our best result here. Its wiki-augmented counterpart scores highest on dev/test (Table 1) but slightly lower on the Blind Test set (87.30) – echoing NoPnx-PA, where the ensemble (our only tuned dev/test comparison point) is again not the Blind Test set winner. On a controlled, same-checkpoint comparison (Appendix B.1), ensembling beats every individual backbone on every subtask, with the gain growing as structural cues are removed; a document-level bootstrap puts the resulting noise floor at roughly 0.8–1.3 F_1 points, so several of the gaps discussed above are not far outside it.

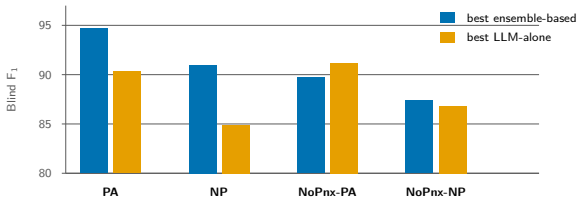


Figure 4: Best Blind F_1 per subtask, ensemble-based vs. LLM-alone systems (Table 2). The ensemble's advantage shrinks and then reverses as punctuation and paragraph cues are removed (PA → NP → NoPnx-PA), where the LLM overtakes it – the clearest single view of this paper's genre/cue-dependence claim (Section 1, Appendix A).

6 Conclusion

As motivated in the Introduction, sentence-boundary judgment on AraSeg 2026 is genre-dependent, and our results show what that costs each modeling paradigm. A supervised ensemble that directly learns boundary statistics from labeled examples is the stronger, more consistent choice on dev/test across all four subtasks (Table 1), and reaches the top Blind Test set score for PA and NP too (Table 2; single-run estimates, std 0.11 F_1 on a 3-seed check, Appendix B). NoPnx-PA is a partial exception: our LLM system's original seed prompt beats both the ensemble and the ensemble+LLM blend on the Blind Test set – the one case where combining did not help, since the LLM only overrides tokens where our two voting backbones disagree (Section 3.2). On NoPnx-NP, stripped of ev-

ery structural cue, the LLM is competitive enough that we combine it with the ensemble for our open-track entry (Section 3.2).

7 Limitations

This paper's Blind Test set comparisons (Table 2) are single-run estimates; our only variance evidence is a 3-seed check on one single-backbone configuration (Appendix B, std 0.11 F_1) and a document-level bootstrap noise floor of roughly 0.8–1.3 F_1 points on the labeled test split (Appendix B.1); several of the gaps we discuss, especially NoPnx-PA's and NoPnx-NP's, are not far outside that floor. The genre-stratified breakdown (Appendix A) uses a smaller, faster LLM (`gemini-3.1-flash-lite-preview`) than the one that produced our official Blind Test set scores (`gemini-3.6-flash`), so its absolute numbers are not directly comparable to Table 2 and only the genre-relative pattern should be read from it. Our GEPA searches (Appendix E) ran on small internal validation splits (30–80 documents, sometimes disjoint from the official dev split) rather than the full 222-document dev set, so their margins are noisy and, for PA and NoPnx-PA, did not transfer into an adopted prompt. NoPnx-NP's open-track system depends on pseudo-labels from a keyword-bucketed Wikipedia sample whose genre-bucketing accuracy and label quality we did not audit. Finally, our LLM system depends on a commercial, closed-weight API whose behavior and availability we do not control, which limits the exact reproducibility of these results going forward.

8 Ethical Considerations

This system performs low-level sentence-boundary segmentation; it does not generate, moderate, or interpret content, and carries minimal potential for misuse. The corpus (Section 2) includes religious genres (Qur'ānic verses, Biblical passages, hadith); our systems treat this text purely as a structural segmentation target and make no claims about religious interpretation or authority. The open-track Wikipedia augmentation (Section 3.2) uses only public, freely licensed text, adding no personal or sensitive data beyond what the shared task's own corpus and Wikipedia already contain. The commercial LLM API used (Section 3.3) was accessed under its provider's standard terms; no human subjects were involved in this work.

References

Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. 2025. GEPA: Reflective prompt evolution can outperform reinforcement learning. *Preprint*, arXiv:2507.19457.

Wissam Antoun, Fady Baly, and Hazem Hajj. 2020. AraBERT: Transformer-based model for Arabic language understanding. In *Proceedings of the 4th Workshop on Open-Source Arabic Corpora and Processing Tools, with a Shared Task on Offensive Language Detection*, pages 9–15, Marseille, France. European Language Resources Association.

Wissam Antoun, Fady Baly, and Hazem Hajj. 2021. AraELECTRA: Pre-training text discriminators for Arabic language understanding. In *Proceedings of the Sixth Arabic Natural Language Processing Workshop*, pages 191–195, Kyiv, Ukraine (Virtual). Association for Computational Linguistics.

Mohamed Basem, Mohamed Younes, Seif Ahmed, and Abdelrahman Moustafa. 2025. !MSA at BAREC shared task 2025: Ensembling Arabic transformers for readability assessment. In *Proceedings of The Third Arabic Natural Language Processing Conference: Shared Tasks*, pages 297–305, Suzhou, China. Association for Computational Linguistics.

Timothy Dozat and Christopher D. Manning. 2017. Deep biaffine attention for neural dependency parsing. In *Proceedings of the 5th International Conference on Learning Representations (ICLR)*.

Mohammed Elkholy, Khalid N. Elmadani, Nizar Habash, and Bashar Alhafni. 2026. Arabic sentence segmentation across genres and punctuation conditions. *Preprint*, arXiv:2606.08025.

Go Inoue, Bashar Alhafni, Nurpeiis Baimukan, Houda Bouamor, and Nizar Habash. 2021. The interplay of variant, size, and task type in Arabic pre-trained language models. In *Proceedings of the Sixth Arabic Natural Language Processing Workshop*, pages 92–104, Kyiv, Ukraine (Virtual). Association for Computational Linguistics.

Pavel Izmailov, Dmitrii Podoprikhin, Timur Garipov, Dmitry Vetrov, and Andrew Gordon Wilson. 2018. Averaging weights leads to wider optima and better generalization. In *Proceedings of the 34th Conference on Uncertainty in Artificial Intelligence (UAI)*, pages 876–885.

Xiaobo Liang, Lijun Wu, Juntao Li, Yue Wang, Qi Meng, Tao Qin, Wei Chen, Min Zhang, and Tie-Yan Liu. 2021. R-Drop: Regularized dropout for neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34.

Bryan Lim, Sercan O. Arik, Nicolas Loeff, and Tomas Pfister. 2021. Temporal fusion transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, 37(4):1748–1764.

Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. 2023. Automatic prompt optimization with "Gradient descent" and beam search. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7957–7968, Singapore.

James Pustejovsky and Amber Stubbs. 2012. *Natural Language Annotation for Machine Learning: A Guide to Corpus-Building for Applications*. O'Reilly Media.

A Genre-Stratified Performance

The Introduction motivates this task as genre-dependent but the main results (Table 1, Table 2) report only whole-subtask F_1. This appendix reports a genre-stratified breakdown on a single 80-document, genre-tagged pool shared identically across all four subtasks and both paradigms (the supervised ensemble and the winning LLM prompt), drawn entirely from the 222-document *dev* split so the ensemble's numbers come from each backbone's actual saved checkpoint probabilities rather than retraining anything. Because PA/NoPnx-PA and NP/NoPnx-NP score the exact same documents under the exact same genre tags, each pair remains a minimal pair isolating exactly the effect of removing punctuation and paragraph breaks, just at $4\times$ the sample size of an earlier 20-document pilot. There is no open/closed split: each subtask uses its single winning system per paradigm.

A.1 Winning LLM Prompt, All Four Subtasks

Each subtask uses its winning/deployed LLM prompt (Appendix D: the improved prompt for PA, GEPA candidate 11 for NP, the unmodified seed for NoPnx-PA, the hierarchical coarse-to-fine winner for NoPnx-NP; Appendix E traces how the NP and NoPnx-NP prompts were found).

Caveat – not comparable to Table 2. For throughput, this run used `gemini-3.1-flash-lite-preview` rather than `gemini-3.6-flash`, the model that actually produced Table 2's official Blind Test set scores. The absolute F_1 values below are therefore systematically lower than Table 2 and should not be read as a re-measurement of it; what is meaningful is the genre-relative pattern, macro F_1 per document with a doc under each of its genre tags. PA's quiz score (0.258) looks like an outlier but is not a bug: on several of these documents the model places the boundary on the closing parenthesis of a bare letter option marker (`"(a)"`) rather than on the numeric answer that follows it (`"2007"`) – a genuine `gemini-3.1-flash-lite-preview` misapplication of the quiz rule (Appendix D), confirmed by inspecting its raw output; every referenced token still resolved correctly.

A.2 Supervised Ensemble and Its Individual Backbones, All Four Subtasks (Same 80-Document Pool)

This table reports, for the same 80 documents as Table 3, every individual backbone's genre-stratified F_1 (thresholded at 0.5, matching Table 7's convention) alongside the full ensemble's (mean backbone probability at the subtask's re-tuned threshold: 0.49 PA, 0.33 NP, 0.47 NoPnx-PA, 0.41 NoPnx-NP). All four subtasks now use the genuine 3-backbone ensemble (CAM = CAMeLBERT-MSA, ArB = AraBERTv02, ArE = AraELECTRA): NP/NoPnx-PA/NoPnx-NP's CAMeLBERT-MSA backbone was previously missing (its per-document probabilities were never preserved from the original runs, so this table and Table 7 previously reported a 2-backbone AraBERTv02+AraELECTRA proxy for those three subtasks); it was trained fresh with the identical recipe and is included here and in Table 7.

B Single-Backbone Component Ablations

The submitted systems (Section 3.1) report only the full recipe's score. This appendix reports component-level ablations run during development at the single-backbone level; none of these isolate a component's marginal contribution inside the final 3-backbone ensemble itself.

A 3-seed check of the GRN+R-Drop α=2.0 single-backbone (CAMeLBERT-MSA) configuration gave dev F_1 of 94.24 (base seed), 94.11, and 94.37 – mean 94.24, std 0.11. Swapping only the backbone in this same recipe, AraBERTv02 reaches 94.39 dev F_1 versus CAMeLBERT-MSA's 94.24 (+0.15).

B.1 Per-Backbone vs. Ensemble, All Four Subtasks

Table 2's "single-backbone" rows are not a clean ensembling ablation: each is a different, earlier configuration (different head, different regularization) rather than one backbone from the exact same recipe as the submitted ensemble (Section 5). Table 7 instead isolates ensembling itself: each individual

Genre	n	PA	NP	NoPnx-PA	NoPnx-NP
dialogue	11	0.805	0.751	0.829	0.766
Qur'ān	8	1.000	0.991	0.901	0.878
essay	30	0.769	0.760	0.757	0.714
headers	11	0.776	0.696	0.831	0.731
lists	7	0.802	0.716	0.771	0.731
hadith	3	0.607	0.539	0.529	0.674
poetry	1	0.975	0.747	0.990	0.702
song	2	0.952	0.681	0.820	0.601
reported speech	4	0.572	0.720	0.706	0.691
bible	14	0.765	0.715	0.623	0.610
quiz	13	0.258	0.876	0.603	0.605
Overall	80	0.708	0.777	0.725	0.697

Table 3: Genre-stratified macro F_1 of each subtask's winning LLM prompt (`gemini-3.1-flash-lite-preview`; see caveat above), same 80 dev-split documents for all four columns.

		PA				NP				NoPnx-PA				NoPnx-NP			
Genre	n	CAM	ArB	ArE	Ens	CAM	ArB	ArE	Ens	CAM	ArB	ArE	Ens	CAM	ArB	ArE	Ens
dialogue	11	0.951	0.956	0.948	0.960	0.929	0.927	0.927	0.934	0.843	0.870	0.879	0.877	0.822	0.837	0.853	0.849
Qur'ān	8	1.000	1.000	1.000	1.000	1.000	0.996	1.000	1.000	0.805	0.795	0.801	0.809	0.821	0.851	0.804	0.849
essay	30	0.923	0.933	0.926	0.930	0.909	0.909	0.910	0.915	0.851	0.861	0.853	0.865	0.824	0.847	0.849	0.852
headers	11	0.928	0.938	0.924	0.932	0.857	0.877	0.859	0.869	0.854	0.883	0.870	0.884	0.806	0.831	0.835	0.836
lists	7	0.950	0.957	0.959	0.960	0.855	0.883	0.864	0.875	0.855	0.858	0.812	0.868	0.828	0.794	0.830	0.839
hadith	3	0.834	0.786	0.879	0.869	0.803	0.778	0.790	0.826	0.844	0.841	0.696	0.825	0.844	0.779	0.727	0.841
poetry	1	0.985	0.995	0.990	0.990	0.711	0.757	0.721	0.772	0.990	0.995	0.990	0.990	0.723	0.743	0.743	0.762
song	2	0.974	0.979	0.977	0.977	0.663	0.689	0.636	0.686	0.914	0.931	0.943	0.928	0.597	0.765	0.791	0.733
reported speech	4	0.864	0.847	0.848	0.851	0.835	0.845	0.845	0.873	0.806	0.809	0.836	0.831	0.800	0.808	0.838	0.834
bible	14	0.999	0.998	0.999	0.999	0.994	0.997	0.998	0.997	0.725	0.750	0.755	0.755	0.702	0.751	0.741	0.740
quiz	13	0.980	0.988	0.990	0.986	0.959	0.962	0.964	0.964	0.826	0.860	0.870	0.854	0.802	0.846	0.852	0.851
Overall	80	0.951	0.954	0.954	0.956	0.924	0.927	0.924	0.930	0.821	0.837	0.832	0.840	0.795	0.824	0.821	0.829

Table 4: Genre-stratified F_1 of every individual ensemble backbone plus the full 3-backbone ensemble, all four subtasks, same 80 documents as Table 3. CAMeLBERT-MSA is consistently the weakest of the three backbones on every subtask (it is MSA-only pretrained, unlike AraBERTv02/AraELECTRA), most visibly on NoPnx-NP; ensembling still beats every individual backbone in almost every cell despite that. Comparing this table to Table 3: the ensemble outperforms the LLM prompt in nearly every genre/subtask cell, most sharply on PA quiz (0.986 vs. 0.258, see caveat above) and on genres/subtasks where structural cues are weak (e.g. NoPnx-NP hadith, essay); the two paradigms are closest on Qur'ān with punctuation visible (PA, NP), where both are already near-ceiling.

Configuration	Dev F_1	Test F_1
Bare (linear head)	93.26	92.73
+GRN head, weighted CE	94.15	93.56
+GRN+SWA (no R-Drop)	94.09	93.59
+GRN+R-Drop α=0.5	94.14	93.81
+GRN+R-Drop α=1.0	94.26	93.87
+GRN+R-Drop α=2.0 (submitted)	94.24	93.95
+GRN+R-Drop α=3.0	94.25	93.89

Table 5: PA single-backbone (CAMeLBERT-MSA) component ablation; all rows share backbone and data, differing only in the listed component. R-Drop gives a consistent gain over the bare GRN head; SWA alone (no R-Drop) is roughly neutral at this single-backbone level. Note α=1.0 slightly edges out the submitted α=2.0 here (94.26 vs. 94.24 dev) – α=2 was carried forward from earlier tuning and not re-swept after later changes.

Head (NoPnx-NP, single-backbone)	Dev F_1
BiLSTM	79.57
CRF	79.65
GRN (2-layer)	80.62
GRN (1-layer, submitted)	80.88
GRN+MoE+Biaffine (unsupervised)	81.17
GRN+Biaffine+curriculum	81.20
GRN+Biaffine	**81.62**

Table 6: NoPnx-NP single-backbone (CAMeLBERT-MSA+R-Drop α=2) head-type ablation, 212-document dev split. GRN+Biaffine scores highest here, but the submitted 3-backbone GRN ensemble (85.10 dev, real-data-only; Table 1) still exceeds even this best single-backbone head by roughly 3.5 points – consistent with ensembling mattering more than head-architecture choice for this subtask.

backbone's own test-set predictions (thresholded at 0.5, from the exact saved checkpoints behind Section 3.1's recipe) versus the mean-probability ensemble at the subtask's tuned threshold, all on the same 262-document test split, all four subtasks now on the genuine 3-backbone ensemble (CAMeLBERT-MSA's per-token test predictions were originally preserved only for PA; NP/NoPnx-PA/NoPnx-NP's were trained fresh with the identical recipe to fill this gap). These figures are computed independently from this project's saved backbone-level predictions and may differ slightly from Table 1's officially-submitted figures, since checkpoints were retrained more than once during development – the point of this table is the within-table comparison, not an exact reproduction of the submitted numbers.

A document-level bootstrap (2000 resamples,

Subtask	Best single	Ensemble	Gain
PA	94.95	**95.05**	+0.10
NP	93.02	**93.25**	+0.23
NoPnx-PA	88.44	**88.81**	+0.37
NoPnx-NP	85.34	**85.47**	+0.14

Table 7: Best individual backbone vs. 3-backbone ensemble, test F_1, all four subtasks. Ensembling beats every individual backbone on every subtask. NoPnx-NP's gain is smaller with the genuine 3rd backbone included than the 2-backbone proxy previously suggested (+0.14 vs. an earlier +0.33 estimate) – CAMeLBERT-MSA is its weakest backbone by the largest margin of any subtask (Appendix A.2), which caps how much an unweighted average can gain there.

262 test documents) around each subtask's ensemble test F_1 gives 95% confidence half-widths of 0.84 (PA), 1.12 (NP), 1.04 (NoPnx-PA), and 1.28 (NoPnx-NP) points. This is computed on the labeled test split, not the Blind Test set (whose gold labels are not available to us), but gives a concrete noise floor: gaps between systems or subtasks smaller than roughly one point are within the range this bootstrap alone would produce from resampling the same fixed model, before any real difference in behavior.

C Backbone Agreement Analysis (Dev Set)

Table 8 measures token-level agreement between AraBERTv02 and AraELECTRA (the two backbones used in the combination rule of Section 3.2) on each subtask's 222-document dev split, computed directly from each model's saved per-token dev probabilities against gold labels.

As structural cues are progressively removed (PA $\rightarrow$ NP $\rightarrow$ NoPnx-PA $\rightarrow$ NoPnx-NP), the two backbones agree less often, are less reliable when they do agree, and their disagreements concentrate increasingly on true boundary tokens (roughly $9\times$ to $23\times$ the non-boundary disagreement rate, growing with the subtask) – direct, token-level evidence for the cue-dependence claim motivating this paper (Section 1), measured across subtasks rather than across genres specifically. This also substantiates the mechanical explanation in Section 5 for why the LLM tiebreak has limited room to help on NoPnx-PA: the two backbones disagree on only 1.53% of dev tokens, so even a perfect tiebreaker could only revise a small, if disproportionately boundary-heavy, fraction of predictions.

D LLM Prompts

D.1 NoPnx-PA (unmodified seed prompt)

This is the complete, verbatim seed prompt; no GEPA-evolved candidate outperformed it, so it is also our submitted prompt (Section 5).

> **Task.** You are given a document as a flat, ordered list of Arabic words with NO punctuation, but paragraph breaks ARE shown, as literal [PARAGRAPH BREAK] markers in the list. For each word (not the [PARAGRAPH BREAK] markers themselves), decide whether a sentence boundary falls immediately after it (1) or not (0).
>
> **What counts as a sentence boundary.** The gold annotations you are trying to reconstruct were created by first splitting the original punctuated text automatically at punctuation marks, then manually correcting that automatic split: annotators introduced an additional split only when two adjacent spans were actually independent sentences, and merged spans back together when punctuation had incorrectly separated parts of a single sentence. A dependent or incomplete clause is never given its own boundary – only a complete, independent sentence is. Your job is to reconstruct where that manual correction process would have landed, without ever seeing the punctuation itself.
>
> **Paragraph breaks are a clue, not a guarantee.** A [PARAGRAPH BREAK] marker is strong evidence that the word immediately before it ends a sentence, since paragraphs virtually always begin on a sentence boundary. It is not an absolute guarantee, though – apply the same genre-density judgment either way, and do not assume every word right after a [PARAGRAPH BREAK] starts a brand new independent unit if the surrounding content says otherwise.
>
> **Verse and structured-answer overrides.** For Qur'ānic verses, New Testament and Old Testament text, and poetry, the verse is the unit: a sentence boundary falls at the end of every verse, regardless of ordinary clause completeness, and in poetry this same boundary also marks a paragraph break. For question-and-answer material structured like a multiple-choice exam, the question stem is its own unit and every answer option is its own unit, however short.
>
> **Output format.** Return a list of boundary entries, each with the exact word (copied character-for-character) that ends the sentence and its best-guess 1-indexed position (used only to disambiguate repeated words), in increasing order, no duplicates. The last word of the document virtually always ends a sentence.

D.2 NoPnx-NP (GEPA meta-harness winning candidate)

This is the complete, verbatim prompt actually loaded by the Blind Test set inference run (`nopnx_np_hierarchical_coarse_to_fine_winner.md`) that produced the 86.80 Blind Test set result in Table 2; its two-level structure is what

Subtask	Agree rate	Acc. if agree	Disagree rate	Disagree, boundary tok.	Disagree, other tok.
PA	99.56%	99.34%	0.44%	2.48%	0.26%
NP	99.37%	99.05%	0.63%	4.04%	0.33%
NoPnx-PA	98.47%	98.19%	1.53%	6.33%	1.01%
NoPnx-NP	98.18%	97.57%	1.82%	7.62%	1.17%

Table 8: Backbone agreement/disagreement rates and accuracy on the dev split, all four subtasks. The last two columns break the disagreement rate down by whether the gold label at that token is a true boundary or not.

Section 3.3 refers to as the coarse-then-fine pass. Appendix E traces how it was found.

> **Task.** You are given a document as a flat, ordered list of Arabic words with NO punctuation and NO paragraph breaks. For each word, decide whether a sentence boundary falls immediately after it (1) or not (0).
>
> **Procedure: segment top-down, coarse then fine – never judge the whole document in one flat pass.** Judging sentence boundaries over the entire document in one left-to-right pass lets long stretches of text quietly get merged into a single unmarked run, because nothing in the middle "obviously" forces a split when you are holding the whole document's context at once. Avoid this by decomposing the task into two levels.
>
> *Level 1 – coarse discourse blocks.* First, scan the whole document and mark the positions where the discourse itself shifts: a change of topic, a change of speaker, a switch from narration into quoted speech or back, the start of a new anecdote/report/citation, the start of an enumerated list or a Q&A block, or a switch of genre/register. This should produce a small number of chunks, each internally coherent in topic, speaker, and register. Every coarse block boundary is automatically also a final sentence boundary.
>
> *Level 2 – fine judgment within each block.* Take each coarse block one at a time, using ONLY that block's own words as context. Within the block, apply ordinary clause judgment: the gold annotations being reconstructed were created by first splitting the original punctuated text automatically at punctuation marks, then manually correcting that split – a boundary only where two adjacent spans were truly independent sentences. A dependent or incomplete clause never gets its own boundary. If a block still feels too heterogeneous to judge confidently, that is a sign it should have been split further at level 1. Merge the level-2 results across all blocks, in document order, as the final boundary list.
>
> **Verse and structured-answer overrides.** For Qur'ānic verses, Biblical text, and poetry, the verse is the unit: a boundary falls at the end of every verse (each verse is naturally its own coarse block or fine unit), and in poetry this boundary also marks a paragraph break. For multiple-choice-exam-structured material, the question stem is its own coarse block transition and every answer option is its own fine unit within the Q&A block, however short.
>
> **Output format.** Return a list of boundary entries, each with the exact word (copied character-for-character) that ends the sentence and a best-guess 1-indexed position (used only to disambiguate repeated words), in increasing order, no duplicates. The last word of the document virtually always ends a sentence.

E GEPA Search Trajectories

E.1 NoPnx-NP (current system)

The meta-harness search that produced Appendix D's prompt ran on an internal 80-document evaluation subset (distinct from, and smaller than, the official 222-document dev split used everywhere else in this paper). Figure 5 draws the full 2-generation tree; Table 9 gives the same trajectory as exact numbers.

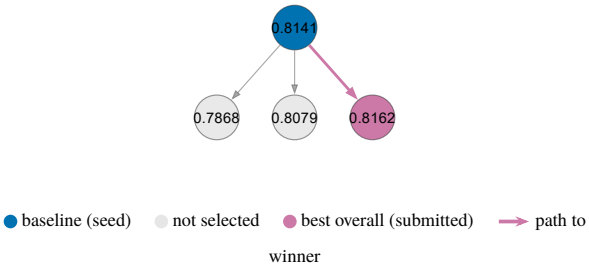


Figure 5: NoPnx-NP GEPA meta-harness tree: baseline (blue) branches into three iteration-2 mutations, two of which regress (gray) and one of which is submitted (purple, path highlighted) – `oversplit_then_merge` (−0.0273), `gap_audit_verification` (−0.0062), `hierarchical_coarse_to_fine` (+0.0022, submitted; full names in Table 9). No per-example Pareto-front data exists for this engine, so unlike the trees below, gray here means simply "not selected," not "off the front."

Iter.	Candidate	Score (Δ)
1	baseline	0.8141 (–)
2	oversplit_then_merge	0.7868 (−0.0273)
2	gap_audit_verification	0.8079 (−0.0062)
2	hierarchical_coarse_to_fine	**0.8162** (+0.0022)

Table 9: Full GEPA meta-harness trajectory for NoPnx-NP. Two of three iteration-2 candidates regressed; the winning margin over baseline is small (+0.0022) on this internal set.

Two of the three iteration-2 mutations *regressed* on this internal set, and the winner's margin over baseline is small. Reading the search's own per-example breakdown, no single candidate dominates:

baseline still wins outright on roughly 32/80 documents, and the direction of error is document-dependent rather than a fixed global bias – candidates that fixed under-splitting on some documents (long, paratactic, hypotactic prose) simultaneously worsened over-splitting on others (short, parallel-clause prose). A planned third iteration, intended to calibrate splitting aggressiveness from each document's own evidence rather than committing to one fixed directional nudge, was never run (search halted after iteration 2; no candidates exist for iteration 3 in the harness's own state log). We report this trajectory in full, including the regressions and the small winning margin, rather than only the winning candidate, since the margin on this 80-document internal set is well within the kind of noise this paper's own Appendix C shows on comparably small samples. Separately, the hierarchical_coarse_to_fine prompt was re-evaluated on a held-out test split disjoint from the search's own 80 documents, where it reached 0.8461 F_1 versus the unmodified baseline's 0.8336 – a gap of similar size to the internal-search margin, which is at least consistent with (though, on one held-out split, not proof against) the win generalizing rather than being an artifact of the search's own evaluation set.

E.2 NP (earlier system, Table 2's "GEPA-distilled prompt" row)

Before the current single-stage, gemini-3.6-flash prompts (Section 3.3), an earlier GEPA search on the two-stage, `gemma-4-31b-it` pipeline (Section 3.3) explored NP prompts with a real, standard GEPA optimizer (reflective mutation with parent-child lineage, not the meta-harness above), on a 30-document internal validation split, and is the origin of Table 2's "GEPA-distilled prompt" row (84.90 on the Blind Test set); it is superseded by the supervised ensemble (91.00) and is reported here purely as a further, real worked example of this project's GEPA usage. Figure 6 draws the full branching tree with node roles taken directly from the GEPA library's own analysis (`gepa.visualization.candidate_tree_dot_from_data`, run against this search's saved optimizer state – not a hand-picked classification), redrawn here in this paper's palette; Table 10 gives the same trajectory, in discovery order, as exact numbers with each candidate's parent and cumulative evaluation-call budget.

Candidates 2, 3, 8, and 11 are all direct children of candidate 1, and their overall scores span nearly the entire observed range (0.4914 to 0.8210) – the same single parent produced both the search's best overall candidate and its lowest-scoring one, which is further evidence (alongside the NoPnx-NP

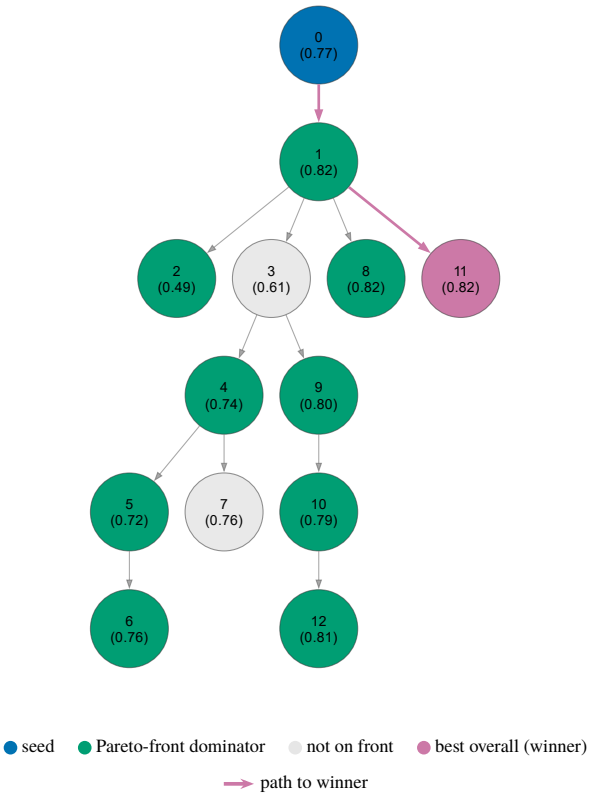


Figure 6: Full GEPA tree for the earlier NP (Gemma) search: 13 candidates, branching to depth 5, roles from the library's own Pareto-front analysis (blue = seed, teal = Pareto-front dominator, gray = not on the front, purple = best overall/winner, path highlighted). Most candidates (teal), including several with a mediocre overall score like candidate 2 (0.4914), are still Pareto-front dominators – each is the single best candidate on at least one validation example, which is exactly why GEPA keeps exploring from them instead of only from the current overall best.

ID	Parent	Score	Calls
0	–	0.7722	0
1	0	0.8184	106
2	1	0.4914	236
3	1	0.6126	296
4	3	0.7449	346
5	4	0.7214	396
6	5	0.7621	446
7	4	0.7631	516
8	1	0.8202	606
9	3	0.7991	676
10	9	0.7935	866
11	1	**0.8210**	976
12	10	0.8100	1086

Table 10: Full GEPA tree for the earlier NP (Gemma) search, in discovery order: candidate ID, its parent candidate, its validation score (30-document internal split), and the cumulative number of LLM evaluation calls spent by the point it was discovered (1086 total). Candidate 11, a child of candidate 1, is the eventual best.

tree above) that a single reflective mutation step's outcome is highly variable and not reliably predictable from its parent's score alone. Yet Figure 6 shows candidate 2 is still a Pareto-front dominator despite that low overall score: GEPA's selection is not purely greedy on the aggregate metric, and a candidate that looks weak on average can still be the sole reason the search keeps a particular failure mode fixed on at least one example.

E.3 PA (Table 2's "GEPA candidate" row)

The same Jul-24 Gemma-reflection GEPA sweep that produced the NP tree above also ran on PA, 17 candidates deep, on its own internal validation split. Candidate 13 (0.8652) is the actual overall best – an earlier script briefly mis-identified candidate 12 (0.7690) as the winner before this was caught and corrected – and it is exactly the candidate behind Table 2's "GEPA candidate" row (85.20 on the Blind Test set), which the separately hand-improved prompt (90.30) beat. Figure 7 draws the full tree, node roles again taken directly from `gepa.visualization`'s own Pareto-front analysis, not a hand classification.

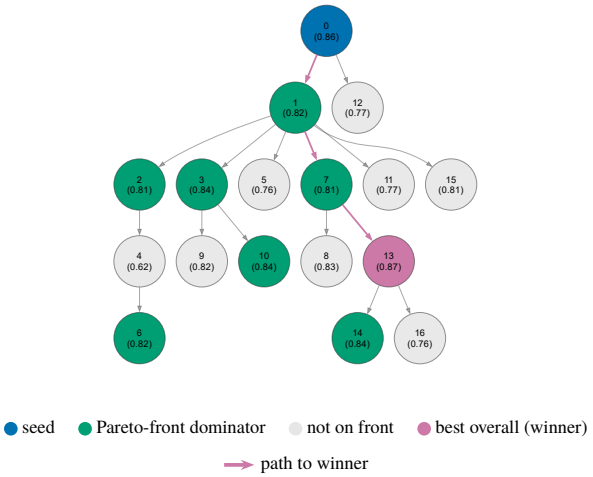


Figure 7: Full GEPA tree for the PA (Gemma) search: 17 candidates (candidate 13, path highlighted). Despite reaching the highest internal score of any PA LLM prompt this project produced, candidate 13 still lost to the manually improved prompt on the Blind Test set (Table 2).

E.4 NoPnx-PA (a separate, later exploration)

Appendix D reports that no GEPA-evolved candidate ever beat the NoPnx-PA seed – that claim comes from an earlier, smaller search (Jul 22) whose only recorded candidate was the seed itself. The same Jul-24 Gemma-reflection sweep as above separately explored NoPnx-PA much further (25 candidates), but on its own internal validation split, not comparable to the Jul-22 run's scale: its seed scored 0.6836 there (vs. 0.9184 on the Jul-22 split) and its best, candidate 12, reached 0.7713 – a real improvement on this run's own terms, but well below the actual deployed seed's score, and never adopted. Figure 8 draws this tree in full since it is real and far more structured than the degenerate one-candidate Jul-22 record, but it is not the search underlying any number in Table 2.

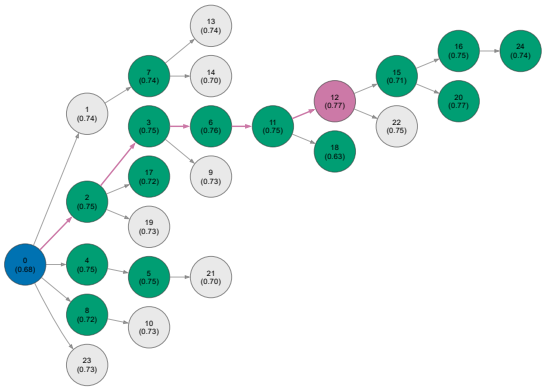


Figure 8: Full GEPA tree for the separate, later NoPnx-PA (Gemma) exploration: 25 candidates, own internal validation scale (see text). Candidate 12 is this run's best; it was tested on the Blind Test set out of curiosity (a genuinely different prompt, worth trying on a stronger model) but no result from it is used anywhere in Table 2.

E.5 Before vs. after GEPA, all four subtasks

Table 11 summarizes seed-vs-best-candidate for the GEPA search actually underlying each subtask's Table 2 entry (NoPnx-PA's row is the real, adopted Jul-22 search, not the larger but unused exploration in Figure 8 above).

F Illustrative Error Patterns

Section 3.3 summarizes three structural fixes found through genre-stratified error analysis. This appendix gives one constructed illustration of each pattern, with Arabic script and its transliteration side by side – these are worked illustrations of the described failure mode, not a verbatim excerpt from one specific document.

Trailing-punctuation attribution. In a sentence ending قال أبو هريرة:... (*...qala Abū Hurayrah:*), the gold boundary falls on the colon-equivalent

Subtask	Seed	Best	Δ	Adopted
PA	0.8628	0.8652	+0.0024	no
NP	0.7722	0.8210	+0.0488	yes
NoPnx-PA	0.9184	0.9184	+0.0000	(seed)
NoPnx-NP	0.8141	0.8162	+0.0021	yes

Table 11: Seed vs. best-candidate internal validation score for each subtask's real, adopted-search GEPA trajectory. Each row's scale is internal to that search (different validation sets/sizes across subtasks, noted in each subsection above) – not comparable across rows, only within. "Adopted" marks whether the best GEPA candidate itself became the deployed prompt: no for PA (a manually improved prompt won instead) and NoPnx-PA (best candidate is the seed – GEPA found no improvement at all), yes for NP and NoPnx-NP. GEPA moved the needle substantially only on NP.

punctuation token itself, not on the preceding word *Hurayrah*. A resolver that always anchors a boundary to the nearest *word* misses this by one token; the fix explicitly allows a punctuation token to be the boundary anchor.

Verse citation vs. MCQ option. *...as stated (2:255) the throne...* is a Qur'ānic āyah citation: the parenthetical is not a boundary. *...Which is the capital? (a) Amman (b) Irbid...* is a multiple-choice option list: each option boundary *is* a real boundary. Both patterns place short parenthetical-adjacent tokens in visually similar positions but resolve in opposite directions (Section 3.3); genre context, not surface form, disambiguates them.

Comma vs. period surface form. A comma-separated list of independent clauses (*...the market closed, the workers left, the street emptied...*) can each be an independent sentence-level unit despite comma punctuation, while some period-adjacent tokens in enumerations or abbreviations are not real sentence boundaries at all – surface punctuation alone under-determines the label, which is why the punctuation-visible subtasks (PA, NoPnx-PA) still need genre-conditioned rules rather than a lookup table from punctuation mark to boundary.